IEEE Access
Multidisciplinary : Rapid Review : Open Access Journal

Received 31 October 2025, accepted 6 December 2025, date of publication 17 December 2025, date of current version 23 December 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3645466

PERSPECTIVE

Vibe Coding as a Reconfiguration of Intent Mediation in Software Development: Definition, Implications, and Research Agenda

CHRISTIAN MESKE[1,2], TOBIAS HERMANNS[1], ESTHER VON DER WEIDEN[1], KAI-UWE LOSER[1], AND THORSTEN BERGER[2]

[1]Faculty of Mechanical Engineering, Ruhr University Bochum, 44801 Bochum, Germany
[2]Faculty of Computer Science, Ruhr University Bochum, 44801 Bochum, Germany

Corresponding author: Christian Meske (christian.meske@rub.de)

ABSTRACT Software development is undergoing a fundamental transformation as vibe coding becomes widespread, with large portions of contemporary codebases now being generated by Artificial Intelligence (AI). The disconnect between rapid adoption and limited conceptual understanding highlights the need for an inquiry into this emerging paradigm. Drawing on an intent perspective and historical analysis, we define vibe coding as a software development paradigm where humans and Generative AI (GenAI) engage in collaborative flow to co-create software artifacts through natural language dialogue, shifting the mediation of developer intent from deterministic instruction to probabilistic inference. By intent mediation, we refer to the fundamental process through which developers translate their conceptual goals into representations that computational systems can execute. Our results show that vibe coding redistributes epistemic labor between humans and machines, shifting expertise from technical implementation toward collaborative orchestration. We identify key opportunities, including democratization, acceleration, and systemic leverage, alongside risks such as black-box codebases, responsibility gaps, and ecosystem bias. We conclude with a research agenda spanning human-, technology-, and organization-centered directions to guide future investigations of this paradigm.

INDEX TERMS Vibe coding, generative artificial intelligence, large language models (LLM), history of software development, human–computer interaction, intent mediation, cognitive work.

I. INTRODUCTION

The software development landscape is undergoing a profound transformation. Organizations that fund and mentor early-stage startups in Silicon Valley report that 25% of companies in their Winter 2025 cohort had codebases that were 95% AI-generated [1]. Further, analyses of enterprise codebases indicates that the adoption of AI-assisted development has substantially increased code creation velocity, with pull requests surging by 70% since late 2022, even as the number of developers has remained steady [2]. Such statistics reflect the rapid emergence of what Andrej Karpathy [3] has termed "vibe coding," a conversational way of creating artifacts where developers "see stuff, say stuff, run stuff" in dialogue with Artificial Intelligence (AI) systems, fundamentally altering how software is conceived and created. Rather than crafting an artifact through code line by line, developers are increasingly enabled to articulate higher-level intentionality through open-ended conversational loops, in which AI not only generates code, but also engages in problem framing and iterative sense-making [4]. While vibe coding shares surface similarities with other forms of AI-assisted development, it represents a qualitatively distinct practice as described by Karpathy. Instead of merely using prompts to generate code snippets, vibe coding describes an entire development process conducted through natural language dialogue. The focus, therefore, lies not on prompt engineering, which concerns the deliberate crafting and

The associate editor coordinating the review of this manuscript and approving it for publication was Derek Abbott.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

refinement of inputs to optimize discrete AI outputs, but on an iterative, conversational workflow in which software artifacts emerge through sustained human-AI exchange. Additionally, traditional co-creative coding assistants presuppose a developer who remains immersed in the codebase, working at the level of syntax and implementation, while the AI offers recommendations that are reviewed and selectively integrated. Vibe coding inverts this relationship: the developer operates at the conversational level, guiding through high-level intent and iterative feedback, while the AI assumes responsibility for implementation.

This shift represents more than a technological convenience; it marks a fundamental reconfiguration of intent mediation in software development ,which we understand as the process of translating conceptual goals into representations that computational systems can execute has been central to software development [5], [6]. For instance, Norman [56] refers to the Gulf of Evaluation and the Gulf of Execution as gaps between user intent and system response, while Leveson [6] demonstrated how system purpose and design principles must be systematically translated into executable representations. Throughout computing history, major paradigm shifts have transformed how humans translate desired outcomes into machine-executable instructions: from the physical manipulation of hardware circuits in systems like Electronic Numerical Integrator and Computer (ENIAC) and Zuse's Z3 [7], [8], through symbolic abstractions like assembly and high-level languages such as Fortran and Algol [9], [10], to object-oriented paradigms and integrated development environments [11]. Each transition fundamentally altered the cognitive demands and epistemic requirements of software development. Especially Nygaard [12] was interested in democratizing the development process, making it more accessible to end-users. Vibe coding can be seen in line with this development, but creates a different kind of shift: from deterministic instruction, where developers must explicitly encode intent through formal syntax, to probabilistic interpretation, where AI systems infer meaning from naturalistic expression and assume responsibility for translating human goals into executable code. This transformation extends far beyond productivity gains, fundamentally reshaping who can develop software [13]. Vibe coding reframes software development as interpretive co-creation, where humans and AI agents collaboratively construct solutions through iterative dialogue and mutual interpretation, rather than formal construction, where developers must explicitly design and execute all implementation details through predetermined syntax and logical structures, aligning with broader theories of distributed cognition [14] and hybrid intelligence, where cognitive work is dynamically shared between developer and AI agents [15]. The implications of the transformation span the reconfiguration of individual cognitive work, the evolution of professional expertise, and organizational structures, as software development shifts from a specialized craft requiring years of technical mastery to a more accessible, conversational approach, where domain knowledge and strategic thinking become more important as implementation skills, while simultaneously introducing risks of technical deskilling, responsibility gaps, and code quality concerns that challenge established software engineering practices.

While surveys show substantial integration of AI assistants into everyday workflows [16], [17], the conceptual understanding of vibe coding remains underdeveloped. Existing research has examined the integration of Large Language Models (LLMs) into software engineering tasks, highlighting practical benefits in code generation and productivity gains [17], [18]. But many of these approaches predominantly view LLMs as subordinate assistants within conventional development paradigms rather than as collaborative partners in a fundamentally new way of creating artifacts. For instance, Gao et al. [18] systematize technical architectures and performance metrics across LLM variants, while setting aside questions of how such models reconfigure the nature of programming in software development and intent mediation itself. This disconnect between widespread tool adoption and conceptual understanding reflects a broader pattern where practical use outpaces theoretical frameworks, creating urgent needs for systematic analysis of this emerging paradigm. We address this gap by providing the first systematic conceptualization of vibe coding as a distinct programming mode and analyzing its implications for software development practice. Our research is guided by two questions:

1) How can vibe coding be defined as a distinct software development paradigm, and how does it reconfigure the mediation of developer intent compared to traditional practices?
2) What cognitive, epistemic, and organizational implications, both beneficial and problematic, arise from vibe coding?

To investigate these questions, in Section II we begin by tracing the evolution of intent mediation in software development since the 1940s, identifying structural and epistemic shifts across the epochs. In Section III, we then define vibe coding in contrast to to traditional software development, articulating its key attributes and interaction patterns, which anchors our analysis of how vibe coding reconfigures cognitive work, expertise, and epistemic agency. Drawing on this conceptual groundwork, in Section IV we synthesize opportunities (e.g., accessibility, democratization, acceleration) and risks (e.g., deskilling, opacity, responsibility gaps) that emerge from the interpretive nature of vibe coding. In Section V, we critically reflect on the findings and outline future research directions. This paper ends with a conclusion and discussion of limitations in Section VI.

II. HISTORY OF INTENT MEDIATION IN SOFTWARE DEVELOPMENT

Intent mediation in software development has evolved significantly over the decades, reflecting changes in both how intent is expressed and how cognitive effort is distributed between

human and machine. This section traces that evolution across nine decades, each marked by a significant evolution regarding the form of mediation and the nature of software development work. Each era concludes with a synthesis that reflects on the dominant patterns and implications for how intent was conveyed during that period. Together, these historical developments, summarized in Table 1 at the end of Section II, provide a basis for understanding how the mediation of intent has shaped and continues to shape the practice of software development.

A. MANUAL TRANSLATION: HARDWARE MANIPULATION TO ALGORITHMIC SPECIFICATION (1940S-1960s)

In the 1940s programmers mediated intent by physically manipulating machine components [19], [20]. On the ENIAC, programs were "constructed" by manually setting switches and connecting patch cables [19], while Zuse's Z3 used punched tape to feed instructions into fixed hardware circuits [7], [21], [22]. Both were tightly coupled to hardware architecture with no separation between logic and machine operation [8], [23]. Each system required its own approach, making programming an inherently machine-specific task [7].

The advent of assembly languages in the 1950s marked the transition from physical manipulation of hardware to using symbolic expressions [19]. Rather than configuring cables or switches, developers were able to use instructions mimicking natural language. Short textual codes, such as 'ADD,' 'MOV,' or 'JMP' were directly mapped to the machine's binary operations [21], [23], [24]. The new layer of abstraction allowed to mediate intent in a language-like form that was more human-readable, easily modifiable, and replicable compared to manual hardware reconfigurations [25]. Despite the use of textual mnemonics, the development process remained closely tied to machine architecture. Each symbolic instruction still corresponded one-to-one with specific hardware actions, still requiring developers to think in terms of memory addresses, Central Processing Unit (CPU) registers, and exact sequencing of low-level operations [26].

From the late 1950s through the 1960s, programming underwent a significant leap in abstraction, moving beyond the one-to-one symbolic mediation of assembly languages. New high-level programming languages like FORTRAN, ALGOL, COBOL and C moved away from hardware-specific encodings [21], [27], instead emphasizing machine-agnostic, higher-order constructs such as loops, conditionals, and functions. These constructs enabled the formal specification of complex procedural logic [21], [28]. Developers could now mediate intent through single high-level statements [21]. For example, a simple loop could be expressed in one concise line in a high-level language, whereas achieving the same in assembly would require manually managing memory addresses, loop counters, conditional jumps, and instruction flow control in multiples lines of code. Compilers provided the required software capabilities that translated these abstract algorithmic statements into the multitude of low-level instructions required for execution on a specific machine [9], [23], [27], [29]. Overall, this era was marked by efforts to formalize the nature and structure of programming languages, defining programming language grammar [10], establishing concepts such as lexical scoping and block-structured control constructs [10], [30], promoting separation of concerns, abstraction boundaries, and systematic decomposition [6].

The foundational era from the 1940s to the 1960s, thus, demonstrates a profound evolution in how developers mediate intent and engage cognitively with computational systems. From the physical manipulation of hardware circuits requiring intimate machine-specific knowledge, through assembly's symbolic mnemonics that maintained one-to-one hardware correspondence, development culminated in high-level languages that enabled abstract algorithmic intent mediation independent of underlying architecture. This progression fundamentally transformed cognitive work from hardware-focused mechanical controlling to conceptual algorithmic thinking, establishing the foundations for programming as an intellectual discipline.

B. CONCEPTUAL MODELING: STRUCTURED PROGRAMMING TO DESIGN PATTERNS (1970s-1990s)

By the 1970s, structured programming had become dominant, with developers writing procedural logic step by step. This proved increasingly tedious and error-prone [31], [32], motivating declarative languages like Structured Query Language (SQL) and Prolog [33], [34] that shifted focus from defining procedures to specifying conditions [35]. Declarative programming inverted the programmer's relationship with the machine: instead of instructing how to compute a result, one specifies the desired outcome [37]. With SQL, for instance, a developer does not define the procedural steps for accessing and comparing data, instead, they write a single formal statement that describes the structure of the result, leaving the execution strategy to the machine [38]. Similarly, Prolog represented a distinct branch of declarative programming known as logic programming, allowing to define a set of logical facts. Computation then becomes a process of machine-automated resolution: the system searches for results that satisfy a query, automatically applying inference steps that were not explicitly spelled out [39]. In parallel, functional programming offered another alternative to procedural expression of intent. Building on the foundations of early languages like Lisp, functional programming languages such as Scheme and Meta Language formalized computation around the concept of mathematical functions [40], allowing intent to be expressed without stepwise manipulation of state. Like declarative programming, it offered a model where developers could describe what should be computed, while abstracting away from how individual steps were executed [41].

By the 1980s, growing software complexity strained procedural code [21], [23]. Object-oriented programming (OOP) emerged to address this by redefining intent mediation around objects, self-contained entities combining data and behavior [42], [42], allowing developers to model real-world concepts as interacting objects rather than global procedures [19], [27], [43]. Languages like Simula, Smalltalk and later C++ allowed developers to define classes, encapsulate state, and structure programs around message-passing between objects [43], [44]. While intent mediation still occurred through structured programming languages with defined syntax and semantics, developers increasingly approached problems not just through fixed sequences of steps, but by thinking in terms of distinct roles and responsibilities within code. Instead of focusing solely on controlling a singular flow of execution, they began to describe systems in terms of how different parts should interact, offering an alternative mental model to procedural logic.

This shift toward expressing intent through conceptual structures continued into the 1990s with the emergence of design patterns that provided reusable templates that structure software systems and communicate underlying intent consistently [45]. First formalized by Gamma [46], design patterns encapsulate proven solutions to recurring problems encountered in software development. They offer developers a shared vocabulary and a set of best practices that make the underlying design intent more explicit and communicable. Design patterns thus mediate intent not only at the level of individual components but across whole system architectures, embedding requirements and domain logic into reusable forms [47], [48]. By formalizing these solutions, design patterns, by design, mediate the developer's intent, ensuring that underlying principles and requirements are consistently understood and implemented [49].

The 1970s to 1990s, thus, marked a turn from purely procedural control toward more expressive design and mediation of intent. As programming languages and paradigms matured, expressing intent became less about operational detail and more about developers shaping conceptual structures. The cognitive work of programming in software development shifted from managing stepwise execution to articulating coherent designs that reflect how developers understand and frame problems. Instead of translating intent into granular instructions, they began shaping code in forms that aligned with their mental models.

C. COLLABORATIVE SYNTHESIS: FROM PREDICTIVE ASSISTANCE TO AI CO-CREATION (2000s-2020s)

The 2000s saw low-code and no-code platforms emerge, enabling users without expertise to build systems by selecting templates, configuring modules, and assembling prebuilt components through graphical dashboards [50], [51], [52]. This abstracted away boilerplate code [54], [55], making development a matter of selecting features and orchestrating workflows through predefined options. Logic was embedded in the interface itself, constraining and guiding what could be expressed [51], [54], [56]. This approach did not replace traditional software development but introduced an alternative model of intent mediation, one where assembly and configuration took precedence over manual authoring and abstraction, shifting some of the responsibility to the machine to anticipate and interpret the developer's intent. While low-code systems expanded participation, traditional programming remained dominant with IDEs providing static code completion based on lexical rules. In the 2010s, recognition that code exhibited statistical regularities similar to natural language [57], [58] enabled machine learning support [59]. Code completion evolved from nearest-neighbor models [60] to Bayesian networks [61], moving beyond static suggestions to actively interpreting developer intent in context. By the late 2010s, neural models further advanced this approach, learning to predict more context sensitive completions like fitting variable names [62], [63]. These models offered completions that were not only syntactically valid but semantically plausible. While intent was still mediated through traditional code, the nature of interaction changed. Developers engaged in a new kind of dialogue with their tools, assisted by systems that could anticipate intent, transforming the development process into a more assisted activity.

In the 2020s, large language models integrated directly into developer workflows through GenAI [64], [65], anticipating intent and proposing syntactically correct, contextually relevant code. This mediation takes two primary forms. The first is in-line assistance, exemplified by tools like GitHub Copilot, which extends the concept of autocomplete from a single keyword to entire multi-line function blocks [64], [66]. As the developer types a comment or a function signature, GenAI offers a complete implementation as "ghost text", which can be accepted, rejected, or modified [67]. The second form is conversational snippet generation. Here, the developer might temporarily leave the IDE to engage in a dialogue with a LLM like ChatGPT, e.g., asking it to "write a function that generates prime numbers" or "generate ideas how to build an app for image filteres" [65]. The developer then acts as a curator of ideas and generated code [65].

This symbiotic interaction profoundly reallocates cognitive work. The primary burden is no longer the meticulous authoring of every aspect of a artifacts logic. Instead, it shifts to more high-level tasks like prompt articulation, expert supervision, and careful integration. The developer's core cognitive work becomes formulating a clear request (prompt) [68], critically evaluating the AI's output for correctness, security, and efficiency, and then weaving that generated output into the larger fabric of the application. The result ceases to be a self-authored artifact and becomes a collaborative piece that was co-created with the GenAI. While this level introduces a form of natural language interaction, it remains firmly grounded in the production of discrete, syntactically-bound code snippets, a crucial

distinction from the more holistic, goal-oriented mediation that would follow with vibe coding.

The 2000s to 2020s thus marked a decisive shift toward assisted and collaborative forms of intent mediation. Beginning with low-code platforms that allowed users to configure systems through graphical assembly rather than manual coding, intent mediation expanded to include new user groups and interaction models. As statistical patterns in code were recognized and exploited, development environments evolved from passive editors into predictive, context-aware assistants. This trajectory culminated in the integration of LLMs that engage developers in a form of co-creation, where intent is expressed not just through code but through natural language prompts and ongoing dialog. Across these developments, the cognitive work of developers transitioned once again: from authoring and structuring logic manually, to orchestrating, curating, and supervising machine-generated contributions. Software development became a mediated activity not just through tools, but through shared agency between human and machine.

III. VIBE CODING: FROM DETERMINISTIC TO PROBABILISTIC INTENT MEDIATION

Building on the historical trajectory outlined in Section II, this section introduces vibe coding as a new software development paradigm that reconfigures how intent is mediated and how cognitive work is shared between humans and machines. Unlike traditional approaches, where developers explicitly encode goals into formal structures, vibe coding centers on interpretive collaboration with Generative AI that infers, adapts, and implements intent expressed in natural language. The discussion is divided into two parts. Section III-A defines vibe coding as a conversational, multimodal software development paradigm, marked by co-creative timing and semantic-level abstraction. Section III-B examines how this model reshapes cognitive demands and redistributes development expertise, proposing a new configuration of epistemic agency.

A. DEFINITION AND CONCEPTUALIZATION OF VIBE CODING

The notion of "vibe" in software practice evolved from informal discourse among developers to describe collaborative rhythm and cognitive alignment in AI-assisted work. When Andrej Karpathy [3] formalized this as "vibe coding" in early 2025, he crystallized what had become a recognizable interactional pattern, one that has quickly gained traction in both developer communities and industry media [3], [69]. In this sense, vibe coding represents the latest stage in a longer evolution from structured programming and pair programming toward conversational, co-creative modes of software production. LLM-powered IDEs such as GitHub Copilot, Amazon CodeWhisperer, Tabnine, and specialized agents (e.g., Replit's Agent, Devin, Claude Code) are central to the vibe coding workflow. Unlike traditional workflows, vibe coding allows users to "see stuff, say stuff, run stuff" [3] in conversational flow [70], prioritizing intuitive expression over technical specification [71] through probabilistic generative programming.

Notably, Y Combinator [1], renowned startup incubator and venture capitalist from Silicon Valley, reported that 25% of startup companies in its Winter 2025 batch had codebases that were 95% AI-generated, reflecting a move toward AI-assisted development. To understand the significance of this paradigm shift, we must first examine the term itself.

"Vibe" in contemporary discourse refers not only to an ambient emotional atmosphere, but also to a state of resonant interaction: Merriam-Webster [72] defines it as "a distinctive feeling or quality capable of being sensed", highlighting its subjective and relational nature. Colloquially, to "vibe" refers to aligning and harmonizing with another entity, where interaction feels effortless and flow emerges naturally. This phenomenon parallels the concept of networked flow, where creative collaboration flourishes when participants experience strong social presence and collective immersion, enabling seamless idea generation and shared understanding [73]. Similarly, effective teamwork depends on establishing harmony and rhythm across learning modes, which enhances cohesion and drives innovation [74]. These insights suggest that "vibing" is more than a colloquialism: it describes a critical collaborative dynamic in which resonance and synchronization foster successful outcomes. This cultural context illuminates why "vibe coding" aptly captures the essence of this new approach: it emphasizes a synchronous co-creative dialogue where developer and AI find a collaborative rhythm, developing solutions through iterative conversation rather than precise technical specification. These transformations mark a transition from deterministic to probabilistic intent mediation. Accordingly, we define vibe coding as follows

> *Vibe coding is a software development paradigm where humans and Generative AI engage in collaborative flow to co-create software artifacts through natural language dialogue, shifting the mediation of developer intent from deterministic instruction to probabilistic inference.*

Vibe coding manifests through five key attributes: (1) goal-oriented intent expression describing goals rather than implementation; (2) rapid dialogic interaction replacing write-compile-test loops with conversational feedback; (3) implementation abstraction enabling deployment without full understanding of details; (4) dynamic semantic refinement where requirements evolve through AI interpretations; and (5) co-creative flow states establishing productive rhythm between developers and AI. Figure 1 represents these attributes through a simplified example, where a user enters a dialogue with a Generative AI with the aim to vibe-code a website.

While traditional development environments passively expected explicit commands, vibe coding introduces AI as an epistemic agent that, in response to human intention and

TABLE 1. Intent Mediation in Software Development (1940s–2020s).

Era	Anchor	Form of Intent Mediation	Cognitive Work
1940s	Hardware Control	Physical manipulation of switches, plugboards, and wires.	Translating logic directly into a physical machine configuration.
1950s	Symbolic Code	Textual mnemonics representing machine opcodes.	Meticulously managing CPU registers and memory addresses.
1960s	High Level Languages	Structured, high-level textual syntax with formal grammar.	Designing step-by-step logic and managing the state of variables.
1970s	Declarative Paradigm	Domain-specific, descriptive statements that define desired results.	Specifying the "what" and delegating the "how."
1980s	Object Oriented Programming	Mapping of real-world entities that embody roles and responsibilities.	Conceptual modelling of real-world entities.
1990s	Design Patterns	Usage of templates that carry underlying intent.	Identifying common problems and applying proven solutions.
2000s	Component Configuration	Assembling and configuring pre-built visual components.	Orchestrating systems by shaping behavior through interface constraints.
2010s	ML Predictive Assistance	Partial single code lines interpreted by ML.	Reviewing, editing, and integrating context-sensitive completions.
2020s	LLM Code Generation	Single code lines, code-contexts, and natural language prompts interpreted by LLMs.	Prompting, reviewing, and correcting contextually generated code snippets.

interaction, actively participates in knowledge construction, interpretation of requirements, and collaborative sensemaking. Rather than merely executing predefined instructions, it participates in the interpretation of the developer's explicit or implicit goals based on inference, predicting potential needs, and offering output informed by patterns learned across vast code repositories. This epistemic dimension fundamentally reshapes the developer-machine relationship. The AI becomes capable of contributing solutions that may exceed the developer's technical knowledge, identifying ambiguities in requirements that need clarification, and recommending alternative implementation strategies based on its extensive pattern recognition capabilities. These dynamics set the stage for our next section, which examines concrete examples of vibe coding and explores the resulting changes in cognitive work and technical expertise.

Figure 2 illustrates the paradigm shift from deterministic intent mediation in traditional software development to probabilistic intent mediation in vibe coding. In both scenarios, the human actor begins with a specific intent. In our example, the developer carries the intent "I want to sort the list [3, 1, 2] from smallest to largest." (1). To enable computer execution, the developer must first overcome the intent mediation gap between the human and the computational system. Traditionally, this mediation requires the adherence to a rigid and narrow specification space. The developer must produce code that conforms exactly to predetermined syntactic and semantic rules for deterministic, instructional execution (2). In contrast, vibe coding allows developers to bridge this gap through natural language communication, which operates on probabilistic-interpretive principles. The developer mediates intent through interaction with a LLM (3), which assumes responsibility for interpreting the natural language specification and producing executable code outputs (4). Regardless of the mediation pathway, whether through direct deterministic coding (2) or LLM-interpreted probabilistic communication (3), the resulting code output undergoes deterministic processing (e.g., compilation) before execution by the processor, maintaining the same final computational determinism in both paradigms (5).

B. RECONFIGURING COGNITIVE WORK AND EXPERTISE

Building upon the exploration of vibe coding as a collaborative, natural language, dialogic co-creation flow, this section examines how this emerging approach fundamentally reconfigures cognitive demands and expertise in the software development practice. The foundation begins with "cognitive alignment," where mental models and artifacts emerge through natural language, which enables a new "cognitive work division" as tasks redistribute between human and AI partners. This redistribution naturally alters the "cognitive rhythm" of development work, so we argue, creating more fluid patterns of engagement. These changes in turn reshape the "nature of expertise" from technical mastery toward collaborative orchestration. As expertise evolves, "knowledge boundaries" become more permeable, with AI compensating for human knowledge gaps. Finally, these shifting boundaries necessitate reconsideration of "epistemic agency and responsibility" when artifact creation becomes a shared human-AI endeavor. By examining these dimensions, we reveal how vibe coding creates a fundamentally different cognitive landscape for developers. Table 2 at the end summarizes Section III-A and Section III-B by systematically contrasting traditional programming with vibe coding.

Traditional programming often forces developers to translate high-level goals into low-level code, creating a

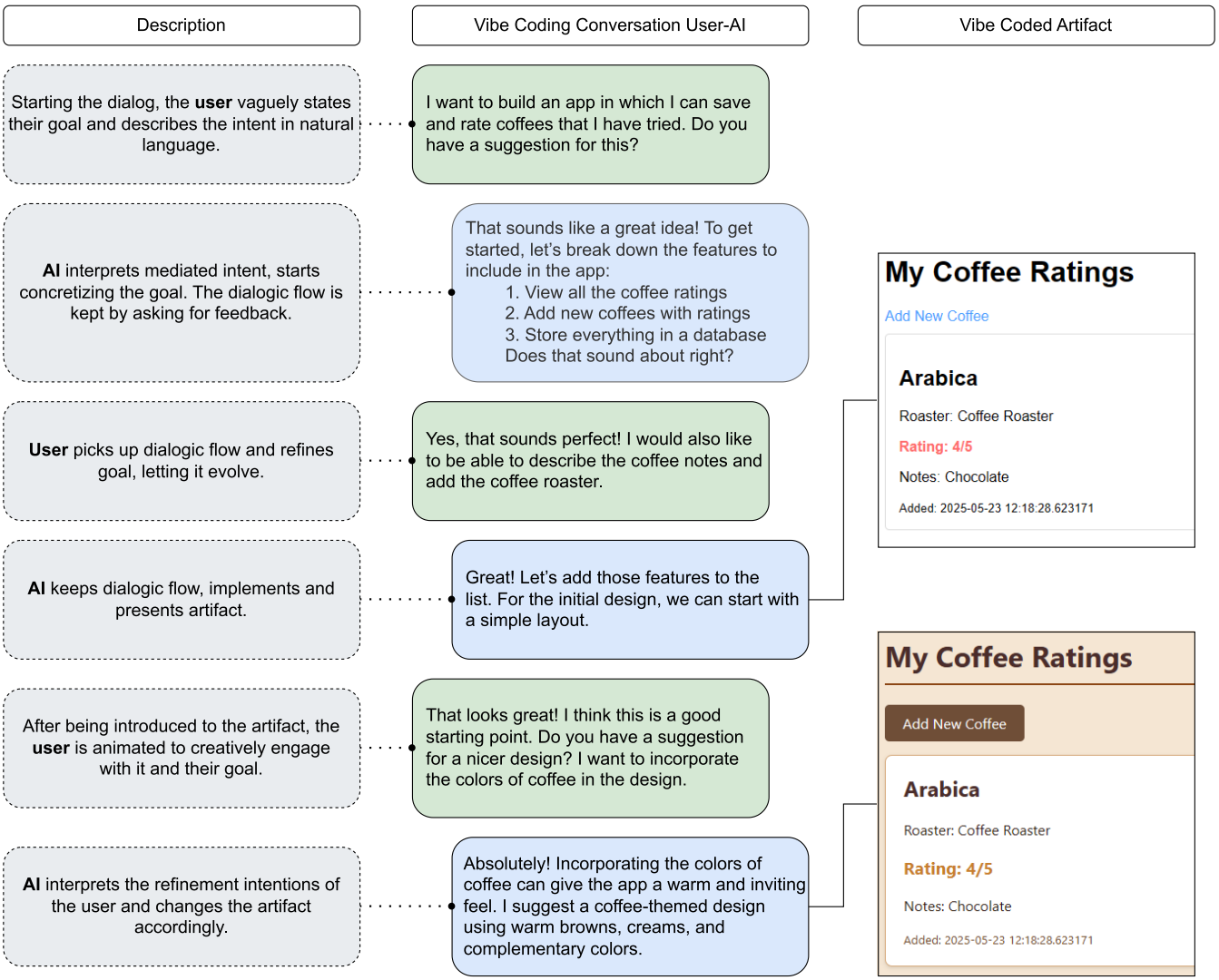

FIGURE 1. **Dialogue between human user (green) and AI (blue) illustrating a vibe coding process.**

disconnect between mental models and formal structures [75]. Vibe coding, by contrast, enables intent expression in natural language, aligning cognitive representation and emerging solutions while reducing extraneous cognitive load [75], [76], [77]. This process establishes closer coupling between the developer's mental model and the evolving artifact, reflecting design principles that favor matched mental models and immediate feedback [78], [79]. The alignment task shifts from API memorization to orchestrating a shared workflow that narrows the gap between intent and code structure, making cognition fundamentally interactive [80].

Vibe coding transforms cognitive work from siloed individual burden to conversational co-creation, exemplifying distributed cognition across human and machine agents [14]. Developers articulate intentions in natural language, expressing goals and concepts while AI handles implementation, freeing cognitive resources for architectural decisions and problem conceptualization. By combining human creativity and strategic thinking with AI's recall and pattern implementation, the complementary strengths of both agents form a more efficient cognitive system than either could achieve alone, a concept referred to as "Hybrid Intelligence" [15]. Also, we argue that Vibe coding creates co-creative flow states with alternating leadership. Developers set goals and interpret results rather than writing every line of code. This resembles cognitive apprenticeship [81], where system feedback partially reveals the AI's "thought process" and enables collaborative learning. Cognitive roles thus shift from a single expert to a human-AI team, with humans as 'vibe directors' and semantic curators while AI serves as a dynamic problem-solver.

This redistribution alters temporal patterns of development as human-AI interactions become critical [82]. While traditional development imposed discrete write-compile-test-debug cycles fragmenting attention, vibe coding creates fluid, conversational cadence enabling faster feedback through

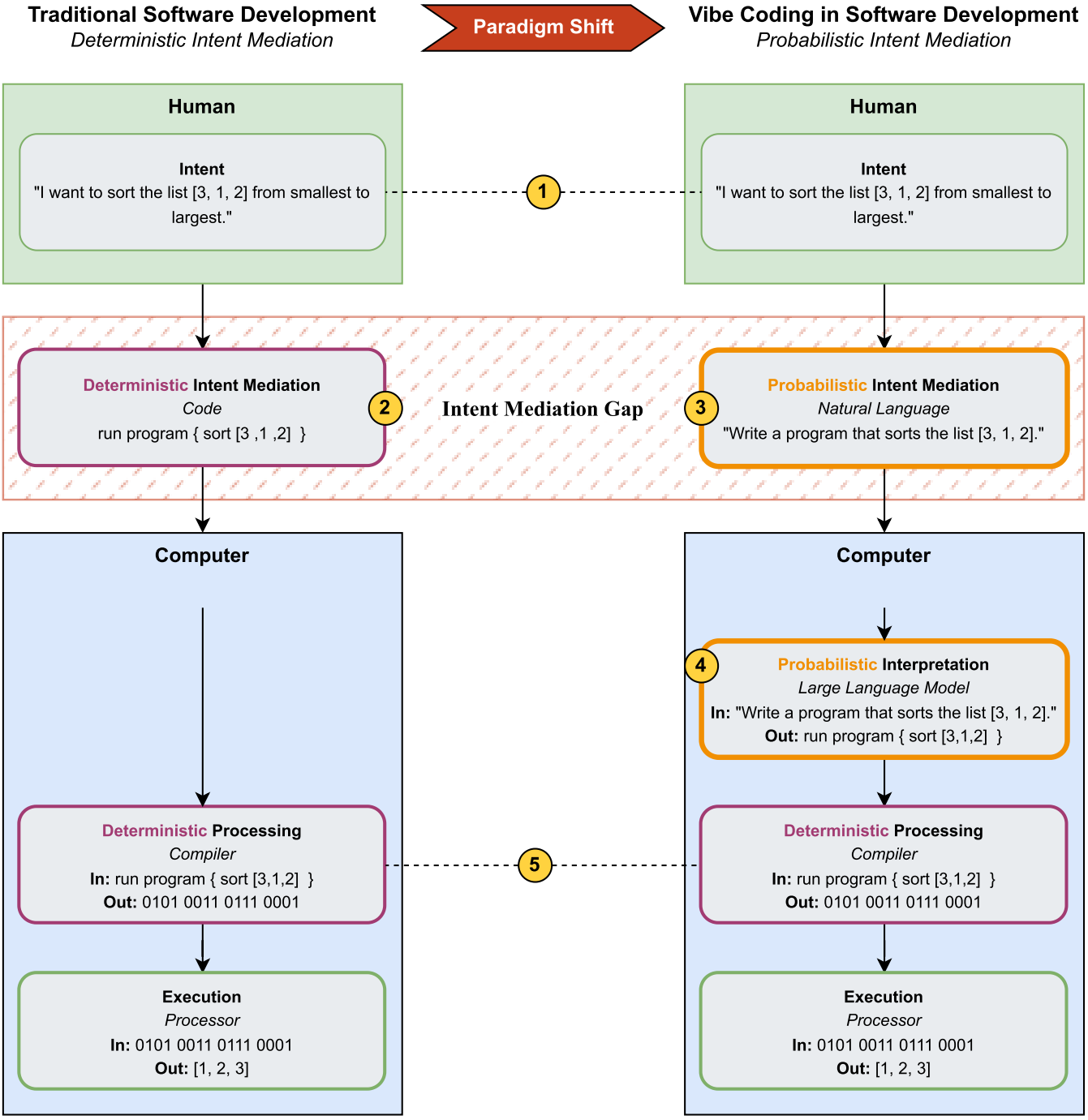

FIGURE 2. Paradigm shift from deterministic to probabilistic intent mediation in software development.

dialogue-based iteration during ideation and design phases. While compile and execution cycles remain necessary for many applications, the conversational nature of the interaction can reduce cognitive fragmentation. Developers can make conceptual pivots more fluidly, keeping their focus on problem-solving rather than implementation details [83]. This iterative engagement aligns with agile principles [84], while vibe coding's emergent semantic alignment and acceptance of partial comprehension allow work to progress even when details aren't fully specified. Cognitive work becomes mutually adaptive, blurring the boundary between ideation and implementation.

Traditional expertise is largely procedural: experts internalize idioms, mentally simulate execution, and excel in domain decomposition [85], [86], [87]. With AI assistance, tacit knowledge shifts to the tool. Vibe coding redefines

expertise as adaptive collaboration, valuing problem framing, output validation, and design thinking. The expert becomes an orchestrator of vibes, steering co-creation. They elicit appropriate AI behavior and embrace evolving code. This connects to Polanyi's tacit knowing: developers "know more than they can tell" while AI generates code [86]. Expertise becomes metacognitive: asking right questions and interpreting answers. This favors technical knowledge blended with conversational skill. In vibe-oriented flow, developers navigate solutions through intuition, resembling jazz improvisation [83], [86]. Development expertise shifts from craftsmanship to synergistic problem-solving. The artifacts talk-back of the artifacts become more instant [79].

Traditional programming required comprehensive expertise in syntax, algorithms, and frameworks developed through years of practice [88]. Vibe coding transforms these boundaries by creating conversational ecosystems where AI compensates for human knowledge gaps, offsetting limited technical understanding [87]. This complementary relationship allows novices to produce functional code despite knowledge deficits, as AI interprets intentions and reduces cognitive load through natural dialogue rather than formal specification. The boundary in vibe coding changes from non-negotiable technical mastery to a permeable threshold where even partial understanding becomes acceptable. While Ericsson et al. emphasize how deliberate practice builds expertise within fixed domains [85], vibe coding expertise involves effectively guiding AI tools across flexible boundaries. The boundary's focus moves futher and further away from deep syntax knowledge toward problem decomposition, domain reasoning, and prompt engineering skills. Developers operating within the vibe coding paradigm still benefit from domain knowledge but face less rigid boundaries around language mastery. Critical thinking and effective AI collaboration become more crucial than memorizing syntactic details, as the complementary AI system handles boilerplate logic and compensates for technical knowledge gaps.

Finally, as knowledge boundaries become more permeable and expertise redistributes, questions of agency and responsibility emerge as critical considerations. Hence, aspects of epistemic agency and responsibility become central as vibe coding transforms knowledge ownership in software development. While traditionally developers were sole epistemic agents, vibe coding creates shared knowledge production. This resulting diffusion of authorship creates potential responsibility gaps [89]: when bugs arise, is the developer or tool at fault? The relational nature of agency in human-AI systems [90] complicates traditional notions of responsibility, as agency emerges from the interaction rather than residing in either the human or AI alone. Human developers retain ultimate agency but work with AI. Studies show humans tend to either over-rely on or under-utilize automated aids [91]. Despite accepting partial comprehension, developers must maintain skepticism by questioning AI suggestions. As Naeem & Hauser [92] note, users can integrate AI while maintaining responsibility. The dialogue-based workflow supports this by making AI "reasoning" partially explicit and editable. While knowledge creation becomes shared, developers transition toward orchestrator roles, remaining gatekeepers responsible for ensuring correctness, security, and goal alignment [89], [91].

The transformations in epistemic agency and responsibility highlighted above represent the culmination of vibe coding's reconfiguration of software development practice across multiple cognitive dimensions. These shifts, from sole to shared knowledge production, from complete comprehension to accepted partial understanding, and from individual to distributed responsibility, fundamentally alter how developers engage with code creation. Table 2 synthesizes these cognitive reconfigurations alongside the intent mediation transformations explored in Section III-A, providing a systematic comparison between traditional software development and the emerging vibe coding paradigm.

IV. OPPORTUNITIES AND RISKS

As our analysis illustrates, vibe coding represents a paradigmatic change in how humans interact with systems to create software, fundamentally reconfiguring the relationship between human intent and machine execution through probabilistic rather than deterministic mediation. This transformation extends beyond productivity gains, reshaping who can develop, how programming tasks are approached, and the organizational structures that result. Drawing on the dimensions of intent mediation, cognitive work, and expertise outlined in Table 2, we further theorize about vibe coding's implications, positing that this evolution presents significant opportunities as well as risks that reflect a complex interplay that likely only hints at the broader consequences.

A. OPPORTUNITIES OF VIBE CODING

The shift toward probabilistic intent mediation in software development creates opportunities ranging from individual empowerment to organizational transformation. By democratizing development, enabling cognitive liberation, accelerating feedback loops, and providing systemic leverage, vibe coding changes not only how software artifacts are produced but also who can participate in digital creation. These opportunities span multiple dimensions of development practice, suggesting that the conversational paradigm will profoundly impact both the social and technical fabric of software development, with many consequences still emerging.

1) COGNITIVE ACCESSIBILITY AND INCLUSION

Natural language and multimodal interfaces are transforming software development from specialized craft to expressive medium accessible to broader, more diverse participants.

TABLE 2. Traditional Software Development vs. Vibe Coding.

Category	Dimension	Traditional Software Development	Vibe Coding
Intent Mediation	Intent Layers	Intent expressed in formal programming languages; limited to text mode via code editors.	Intent expressed through natural language, voice, visual cues; supports multimodal interaction across chat, speech, or graphical interfaces.
	Intent Translation	Human decomposes intent into semantics and syntax; compiler handles syntax-to-execution.	AI infers semantics from naturalistic input and generates syntax; developer guides through prompt iteration and review.
	Intent Fidelity	High fidelity through explicit specification and manual control.	Variable fidelity; AI interpretation introduces ambiguity, requiring testing and refinement.
Cognitive Work	Cognitive Alignment	Requires structured, abstract reasoning aligned with machine logic and formal languages.	Aligns with intuitive, expressive reasoning; supports informal articulation of goals.
	Configuration of Cognitive Work	Developer as sole constructor and debugger; responsible for all formalization.	Developer as articulator, critic, and tester; shares generative and interpretive labor with AI.
	Cognitive Rhythm of Engagement	Linear and staged: plan → implement → test; feedback is delayed and tool-mediated.	Dialogic and iterative: prompt → interpret → revise in near real time; continuous co-adaptation.
Expertise	Nature of Expertise	Emphasizes formal implementation and optimization.	Emphasizes articulation, prompting, validation, and strategic steering of generative systems.
	Knowledge Boundary	Fixed and comprehensive; requires mastery of syntax, algorithms, architecture, and system interaction with minimal external compensation.	Fluid and complementary; AI capabilities offset human knowledge gaps through iterative dialogue, reducing need for technical mastery.
	Epistemic Agency and Responsibility	Human holds full authorship and explanatory authority over code behavior.	Epistemic agency is shared; AI proposes logic, human accepts, tests, and assumes partial accountability.

By enabling communication through everyday language and intuitive interactions, these paradigms lower barriers to entry and foster inclusive innovation. This reconfiguration of the landscape opens several pathways for inclusion.

By enabling AI to infer intent from natural language, domain specialists can directly translate their expertise into functional software. This *empowerment of domain expertise* reduces the reliance on formal programming skills and rebalances traditional technical hierarchies [93].

A *lowered entry threshold* allows individuals without technical backgrounds to participate meaningfully in software development by programming that aligns with intuitive reasoning and thus broadening access and democratizing technological creation [94].

AI's ability to complement human knowledge enables users to make progress even with incomplete technical understanding. This *cognitive scaffolding* supports productive work without requiring full mastery of implementation details [95].

2) COGNITIVE LIBERATION

Vibe coding reframes development from technical execution to strategic orchestration through collaborative dialogue that shapes both problem and solution. This privileges iterative refinement and higher-order thinking over predetermined steps. As a result, vibe coding supports a more reflective and adaptive form of problem-solving, where knowledge is constructed collectively and solutions emerge through interaction and negotiation.

With AI's ability to infer intent from naturalistic input, *rapid prototyping and iteration* is made possible. This accelerates the creation and exploration of initial solutions, enabling rapid testing and refinement of alternative approaches before committing to a final implementation [96], [97].

Enabling *Human-AI Co-Creation* by shifting cognitive work from individuals to collaborative human-AI configurations, allows developers to act as articulators, critics and testers. This partnership frees human resources for strategic and creative tasks while AI manages implementation complexity [98].

As programming emphasizes articulation, prompting, and strategic steering over implementation, *new forms of expertise* emerge. Evaluative and dialogic skills gain prominence over low-level mastery. This expertise evolution creates space for differently skilled individuals to contribute meaningfully to software development [99].

Within the dialogic and iterative development rhythm, requirements evolve through AI interaction rather than upfront specification, thus necessitating *emergent problem understanding*. This emergent approach encourages exploratory design thinking and reduces cognitive overhead in early development stages [99].

By *expanding creative horizons*, freeing from the constraints of formal syntax and implementations, developers

can Within the dialogic and iterative development rhythm, requirements evolve through AI interaction rather than upfront specification [100].

3) ACCELERATED DEVELOPMENT CYCLES

Traditional software development typically progresses through linear stages such as requirements gathering, design, implementation, testing, and deployment. These handoffs often create bottlenecks and limit opportunities for feedback and revision. In contrast, the continuous co-adaptation inherent in vibe coding enables more fluid approaches to system construction.

AI's ability to infer intent from naturalistic input accelerates the creation and exploration of initial solutions, enabling *rapid prototyping, iteration*, and refinement of alternative approaches before committing to a final implementation. This revolutionary speed underscores the core concept of Vibe Coding [101].

By automating the translation from abstract requirements to executable code, the *translation overhead* is reduced. Thus, AI minimizes manual mapping and cognitive load for engineers, expediting development and reducing error rates [102].

The experience is enhanced by a *conversational and iterative flow*. Real-time, dialogic engagement with AI fosters a seamless development rhythm, promoting a creative flow and minimizing friction between ideation and realization [103].

The process of vibe coding allows for *flexible, evolving specifications*. Development proceeds through cycles of prompting, interpretation, and revision, enabling solutions to emerge organically without the need for fully specified requirements at the outset [104].

4) SYSTEMIC LEVERAGE

Beyond its advantages for individuals, vibe coding generates transformational impacts at both the organizational and ecosystem levels. These changes help organizations become more agile and responsive to emerging challenges and opportunities. In this way, the opportunities previously identified, such as increased inclusion, enhanced problem-solving, and more flexible system development, are elevated from isolated benefits to strategic outcomes that shape the direction and success of entire organizations and broader communities.

The adoption of AI and conversational workflows fundamentally restructures organizational capabilities, such as *scalable team efficiency*. By amplifying the output of small teams, this approach enables organizations to achieve results that previously required much larger technical departments. This democratizes innovation, empowering startups and smaller units to compete effectively with established industry players [105].

This shift also facilitates a strategic *talent reallocation*. As the nature of cognitive work transforms from implementation to orchestration, organizations can prioritize hiring domain experts and creative thinkers over traditional technical specialists. This optimizes talent deployment and fosters richer interdisciplinary collaboration across the board [106].

Accessible interfaces and conversational workflows allow organizations to pursue projects previously constrained by significant technical complexity or resource limitations, thus broadening the scope of feasible innovation [107].

The synergy of rapid prototyping, reduced translation overhead, and iterative refinement compresses development cycles, leading to accelerated innovation and market responsiveness. This alignment between emergent solutions and market needs enables faster, more relevant innovation, which ultimately provides a stronger competitive edge [108].

B. RISKS OF VIBE CODING

Despite its promising opportunities, vibe coding introduces significant challenges that warrant critical examination as the paradigm shift toward probabilistic intent mediation reshapes software development practice. The risks span from potential erosion of technical expertise and degradation of code quality to the emergence of responsibility gaps and organizational vulnerabilities that threaten long-term sustainability. Real-world incidents emerging from early vibe coding adoption illustrate that these risks often manifest in areas like security, reliability, and autonomy. Addressing these challenges requires proactive strategies for knowledge preservation, quality assurance, strategic planning and governance, which encompasses policies and structures that regulate AI tool use, establish oversight requirements, and assign responsibility for outcomes. Only by anticipating and managing these risks can we ensure that the benefits of vibe coding are realized without compromising fundamental software engineering principles or introducing unforeseen negative consequences.

1) EROSION OF PROGRAMMING EXPERTISE

As AI increasingly mediates programming through naturalistic input, traditional expertise in code manipulation, syntax understanding, and procedural application may become less central, raising questions about how expertise is now formed, maintained, and transferred.

With developers acting more as articulators than implementers, core technical skills, such as algorithmic thinking, debugging, and architectural planning, may atrophy. Studies on Generative AI capabilities suggest that this *deskilling* or "leveling of ability" is a common outcome [109]. Reliance on AI to fill knowledge gaps can consequently weaken a developer's capacity for systematic solution design and independent problem-solving.

The ability to generate functional code and complex artifacts without requiring deep domain understanding fosters a *false sense of competence*, increasing the risk of overlooking critical issues like security and performance. Illustrating security-related risks, research indicates that participants who had access to an AI assistant were more likely to believe they

wrote secure code, suggesting that such tools may lead users to be overconfident about security flaws [110], [111].

As development evolves increasingly toward prompt engineering, traditional and essential avenues for *transmitting tacit knowledge*, such as direct mentorship, pair programming, and rigorous code reviews, are diminished. This impedes the professional growth of less experienced developers, an issue that requires particular attention to be given to professional development and curriculum reform [112]. The long-term negative impact on novice skill acquisition further reinforces this threat [113].

2) CODE QUALITY AND MAINTAINABILITY

Generative AI integration risks undermining collective practices ensuring code stability. Teams rely on shared standards, review processes, and systematic testing to maintain quality and identify errors. As AI produces more code, these collaborative routines may weaken.

Iterative prompting can lead to unpredictable code rewrites, which makes local debugging less effective and introduces *instability* with unanticipated side effects. This prioritization of immediate functionality over sound architecture accelerates technical debt and creates long-term maintenance burdens, a common issue as AI-based systems manage and accrue complexity [114]. The discovery of security flaws in commercial vibe coding platforms underscores security and autonomy risks in practice [115].

Additionally, developers face the problem of *opaque verification and inconsistent patterns*. As traditional reviews and testing protocols are replaced or influenced by AI-driven judgments, quality assurance becomes less transparent and reliable. Highlighting reliability concerns, large-scale security analysis of over 100 language models revealed that only 55 % of AI-generated code was secure, with security performance remaining largely flat over time even as models improved at generating syntactically correct code [116]. This suggests that functional correctness does not guarantee security quality in AI-generated code. Furthermore, code generated through conversational flows often lacks cohesive structure, consistent patterns, and proper documentation, greatly complicating future maintenance. Reflecting reliability challenges, research highlights that AI-generated code, while simpler, is more prone to unused constructs and hardcoded debugging, which contrasts with human-written code that, despite having greater structural complexity, generally exhibits higher maintainability [117].

3) EPISTEMIC AND RESPONSIBILITY GAPS

Shared epistemic agency distances developers from underlying logic and intent. Distributed responsibility makes it difficult to understand, explain, or assign accountability for outcomes when problems arise, complicating ethical oversight.

A salient concern pertains to the *ambiguity surrounding authorship and accountability*. Dialogic, iterative engagement blurs the boundaries of ownership between human and AI, making it difficult to assign responsibility for errors or unethical outcomes, thereby creating significant governance challenges [118], [119].

As developers increasingly rely on AI-generated code, they become detached from the internal logic of their systems. This partial comprehension makes it harder to understand, intervene, or recover during critical failures, particularly when deep system knowledge is essential. The challenges facing AI-generated code include accuracy, contextual understanding, security, privacy, and ethical considerations, necessitating thorough review and testing [120]. This challenge is specifically known as the *AI Black Box Effect* [121].

the continuous co-adaptation workflow leads to the *loss of intent traceability*. This obscurity makes it difficult to connect original requirements to implemented solutions. Without transparent mapping between prompts, revisions, and the final code, future maintainers cannot reconstruct the rationale behind key decisions, even though traceability is known to improve software maintenance quality [122].

The inherent limitations of the technology create *ethical and data protection blind spots*. AI's limited explanation capabilities and reliance on probabilistic mediation increase the risk of undetected ethical breaches and data protection violations. These issues may only become apparent in production environments or edge cases, necessitating careful scrutiny of AI's output regarding security, privacy, and ethical compliance [120].

4) STRATEGIC AND ORGANIZATIONAL VULNERABILITIES

Vibe coding introduces systemic challenges affecting organizational structures and ecosystems. The collaborative nature complicates maintaining consistent processes, enforcing standards, and ensuring clear responsibility, while interconnectedness can amplify problems across teams and organizations.

The convergence of black box codebases, ethical and data protection blind spots, and inconsistent documentation severely undermines a system's auditability and verifiability. This poses significant barriers to compliance in highly regulated sectors such as healthcare and finance. Expert consensus widely acknowledges the need to establish clear responsibility and accountability for the outputs and impacts of AI-enabled systems, making transparency essential for justice and governance [123].

The current AI landscape introduces significant ecosystem bias and tool dependency. AI inference capabilities are stronger for mainstream programming languages, which grants a strategic advantage to some organizations while disadvantaging those using specialized languages. Simultaneously, reliance on proprietary AI tools and their probabilistic mediation introduces economic and infrastructural dependencies. This reduces organizational autonomy and exposes companies to risks like pricing changes, service

discontinuations, or shifting external priorities. These factors, described as a "Matthew Effect," reinforce existing popularity hierarchies among tools, creating competitive distortions and long-term strategic vulnerabilities [124].

As organizational expertise shifts toward prompt engineering and strategic steering, traditional software development skills, such as authoring code, debugging, maintenance, and scaling, risk being undervalued. This misalignment threatens system continuity, particularly when deep technical knowledge is required for critical systems. The future role of software engineers will continue to change, requiring adaptation of the profession to remain relevant and effective in an AI-assisted environment [125].

V. DISCUSSION

A. VIBE CODING AND THE RECONFIGURATION OF INTENT MEDIATION

Vibe coding represents a shift from deterministic to interpretive and collaborative development.

The nature of software development expertise is hence fundamentally transformed. As discussed in Section III, traditional skills such as implementation-specific fluency and syntactic mastery are increasingly supplanted by new competencies, including problem articulation, prompt engineering, and evaluative judgment. Research on live programming environments highlights the importance of immediate feedback on AI-generated code in fostering these emerging skills [127]. Moreover, recent studies emphasize that the widespread adoption of large language models in software engineering amplifies these challenges, raising critical concerns about code quality, explainability, and the urgent need for updated educational and professional practices [128]. This evolution reflects a redistribution of expertise across a collaborative human-AI system.

As outlined in Section III and Section IV, vibe coding presents opportunities for democratization, acceleration, and enhanced cognitive accessibility in software development. However, it also introduces risks, including technical deskilling, code quality issues, and responsibility gaps. Empirical studies indicate that programming with large language models can yield significant productivity gains [17], but may also introduce new forms of systemic fragility. Recent analysis identifies significant challenges inherent to LLM-assisted code generation, including insecure code generation, hallucinated outputs, irreversible actions, vulnerability inheritance, overtrust, and the absence of standardized validation and rollback protocols [111]. These concerns are compounded by the accumulation of technical debt, as AI-generated code often lacks the structure, documentation, and clarity necessary for long-term maintenance, leading to increased costs and making future modifications and debugging significantly more difficult [126]. The discovery of a critical vulnerability in Wix's Base44 vibe coding platform, which allowed unauthorized access to private enterprise applications through simple exploitation techniques, underscores security and autonomy risks in practice, with researchers warning that fast-paced vibe coding platforms may introduce systemic risks to entire ecosystems [115]. Addressing these challenges will require innovative approaches to software development, as well as revised educational and professional practices. For instance, research has shown that the quality of identifier construction directly affects a developer's ability to comprehend and debug code, suggesting that AI-generated code with ambiguous or poorly chosen names may exacerbate these difficulties [129].

Current institutional frameworks, which rely on explicit control and procedural transparency, are destabilized by vibe coding's shared epistemic agency and fluid knowledge boundaries. This misalignment calls for coordinated adaptation across educational, regulatory, and organizational domains to establish coherent frameworks for this new programming paradigm. Furthermore, the digital environments in which software development now occurs may subtly nudge practitioners toward certain approaches, creating choice architectures that institutional frameworks have yet to recognize or address. Regulatory systems, which assume clear lines of accountability, are challenged by the blurred responsibilities inherent in collaborative human-AI development. The opacity of AI decision-making processes further complicates accountability, making explainable Artificial Intelligence essential for maintaining institutional oversight and compliance.

B. FUTURE RESEARCH DIRECTIONS

The shift to probabilistic intent mediation reveals gaps in understanding how Generative AI reconfigures development practice, expertise, and organizational structures. While offering opportunities for democratization and acceleration, vibe coding introduces uncertainties requiring systematic investigation and governance frameworks. The urgency of establishing such frameworks is underscored by rapid AI adoption in business contexts, where organizations must navigate complex governance challenges. Drawing from a sociotechnical perspective, we organize this research agenda around three interconnected domains: the human actors whose cognitive work and professional identities are being reconfigured, the technological systems whose interpretive capabilities shape what can be expressed and built, and the organizational structures that must evolve to accommodate shared epistemic agency.

1) HUMAN-CENTERED RESEARCH DIRECTIONS

Vibe coding transforms how developers think, learn, and define themselves, necessitating rethinking cognitive models, expertise, and pedagogical frameworks while opening possibilities for broader inclusion. The current professional development has a potential to swap over to the field of End-User-Development. As natural language becomes a primary interface to create artifacts, traditional forms of mastery may

be replaced, or complemented, by new types of articulation and collaborative fluency.

1) How can the new environment be used for software development education and what are appropriate didactical approaches for teaching software development?
2) How must software development education be restructured to develop competencies in prompt articulation, semantic validation, and co-creative orchestration?
3) What forms of expertise are emerging in vibe coding, and how should these be systematically recognized, assessed, and certified?
4) How does vibe coding reconfigure the identity of developers, and what are the implications for professional development and recruitment?
5) Does vibe coding measurably lower participation thresholds for underrepresented groups in software development, and how can this potential be institutionalized?

2) TECHNOLOGY-CENTERED RESEARCH DIRECTIONS

Vibe coding's probabilistic processes remain opaque, raising questions around intent fidelity, semantic alignment, and explainability. While multimodal prompts allow flexibility, they introduce ambiguity and variability complicating reproducibility. Research must focus on transparency, accountability, and certifiability for legal, ethical, or safety-critical workflows.

1) How do different LLMs vary in their interpretation of identical prompts, and what are the systemic implications for reliability, reproducibility, and control?
2) What linguistic and modal features ensure high-fidelity intent expression across LLMs, and can these be codified into standardized prompt design guidelines?
3) How can explainability be operationalized in vibe coding environments to render AI inferences interpretable and critique-ready?
4) What technical and procedural standards are required to certify AI-generated code for use in critical domains with legal, ethical, or safety implications?

3) ORGANIZATION-CENTERED RESEARCH DIRECTIONS

Vibe coding reshapes institutional logic, team composition, and accountability structures. Organizations must adapt methodologies, redefine roles, and implement governance protocols for shared human-AI authorship while maintaining critical technical capabilities. Strategic alignment, compliance, and knowledge retention emerge as key concerns.

1) What new roles must organizations establish to manage prompt engineering, semantic design, and AI oversight in software teams?
2) How should software development processes be restructured to support dialogic iteration, emergent requirements, and epistemic ambiguity?
3) How can accountability and legal liability be operationalized in systems where authorship and agency are shared between humans and AI?
4) How can organizations preserve deep programming expertise for system resilience while scaling vibe coding practices operationally?

These research directions reflect the complex reconfiguration of software development as both a technical and social practice. Vibe coding challenges longstanding assumptions about intent, authorship, expertise, and responsibility in software creation. Addressing these challenges will require interdisciplinary collaboration across computing, design, education, organizational science, and ethics. In particular, ethical inquiry should engage with how vibe coding inherits and potentially amplifies concerns already central to AI ethics. Biases latent in training data may propagate undetected into deployed systems, while the layered opacity of AI-generated code, where neither model nor developer may fully comprehend the implementation, complicates meaningful auditing for fairness, safety, or discriminatory effects. Moreover, as software creation becomes increasingly dependent on proprietary AI systems controlled by few corporations, vibe coding raises broader questions about epistemic dependence, the erosion of developer capacity to scrutinize and challenge generated outputs, and the concentration of power over the tools that shape how software is made. A robust socio-technical research agenda, attending equally to technical architectures and to the human conditions, values, and vulnerabilities they encode, is therefore essential not only for understanding vibe coding, but for shaping its institutional, technical, and cognitive future.

VI. CONCLUSION

This paper defines vibe coding as a software development paradigm characterized by natural language dialogue and collaborative flow between humans and AI. Situating it within the historical evolution of intent mediation highlights how shifts in modality reconfigure the human-machine relationship. Addressing the first research question, the paper has articulated how vibe coding distinguishes itself from traditional software development by shifting intent mediation from explicit instruction to probabilistic, goal-oriented dialogue, as reflected in its five key attributes: goal-oriented intent expression, rapid dialogic interaction, implementation abstraction, dynamic semantic refinement, and co-creative flow states.

In response to the second research question, the analysis has explored the cognitive, epistemic, and organizational implications of vibe coding. The opportunities identified include enhanced cognitive accessibility and inclusion, cognitive liberation, accelerated development cycles, and systemic leverage. However, these benefits are accompanied by significant risks, such as the erosion of programming expertise, challenges to code quality and maintainability, epistemic and responsibility gaps, and strategic and

organizational vulnerabilities. While the definition and implications outlined in this paper offer a foundation for understanding vibe coding, they remain open to refinement as the field develops.

Looking ahead, the future research questions proposed here point toward human-centered, technology-centered, and organization-centered research directions. Continued investigation along these lines will be essential for establishing a nuanced understanding of vibe coding, enabling the field to harness its benefits while addressing its inherent risks as this development modality matures.

ACKNOWLEDGMENT

The authors used ChatGPT-4o (OpenAI) and Claude Sonnet 4 (Anthropic) across all sections of the manuscript for language-related support, including proofreading, rephrasing, structuring, or editing of their-drafted text. All content was reviewed by them, who take full responsibility for the final version of the manuscript.

REFERENCES

[1] I. Mehta. *A Quarter of Startups in YC's Current Cohort Have Codebases That Are Almost Entirely AI-Generated*. Accessed: Oct. 22, 2025. [Online]. Available: https://techcrunch.com/2025/03/06/a-quarter-of-startups-in-ycs-current-cohort-have-codebases-that-are-almost-entirely-ai-generated/

[2] I. Nussbaum. (Feb. 26, 2025). *Faster Code, Greater Risks: The Security Trade-Off of AI-Driven Development*. [Online]. Available: https://apiiro.com/blog/faster-code-greater-risks-the-security-trade-off-of-ai-driven-development/

[3] A. Karpathy. *There's a New Kind of Coding I Call 'Vibe Coding' ...*. Accessed: Oct. 22, 2025. [Online]. Available: https://x.com/karpathy/status/1886192184808149383

[4] A. Sarkar and I. Drosos, "Vibe coding: Programming through conversation with artificial intelligence," 2025, *arXiv:2506.23253*.

[5] D. A. Norman and S. W. Draper, *User Centered System Design: New Perspectives on Human–Computer Interaction*. Hillsdale, NJ, USA: Lawrence Erlbaum Assoc., 1986.

[6] N. G. Leveson, "Intent specifications: An approach to building human-centered specifications," *IEEE Trans. Softw. Eng.*, vol. 26, no. 1, pp. 15–35, Jan. 2000, doi: 10.1109/32.825764.

[7] W. B. Fritz, "ENIAC—A problem solver," *IEEE Ann. Hist. Comput.*, vol. 16, no. 1, pp. 25–45, Mar. 1994, doi: 10.1109/85.251853.

[8] T. Zoppke and R. Rojas, "The virtual life of ENIAC: Simulating the operation of the first electronic computer," *IEEE Ann. Hist. Comput.*, vol. 28, no. 2, pp. 18–25, Apr. 2006, doi: 10.1109/MAHC.2006.40.

[9] J. Backus, "The history of fortran I, II, and III," *IEEE Ann. Hist. Comput.*, vol. 20, no. 4, pp. 68–78, Oct. 1998, doi: 10.1109/85.728232.

[10] J. W. Backus, F. L. Bauer, J. Green, C. Katz, J. McCarthy, A. J. Perlis, H. Rutishauser, K. Samelson, B. Vauquois, J. H. Wegstein, A. van Wijngaarden, and M. Woodger, "Report on the algorithmic language ALGOL 60," *Commun. ACM*, vol. 3, no. 5, pp. 299–311, May 1960, doi: 10.1145/367236.367262.

[11] O.-J. Dahl and K. Nygaard, "SIMULA: An ALGOL-based simulation language," *Commun. ACM*, vol. 9, no. 9, pp. 671–678, Sep. 1966, doi: 10.1145/365813.365819.

[12] B. A. Nardi, *A Small Matter of Programming: Perspectives on End User Computing*. Cambridge, MA, USA: MIT Press, 1993.

[13] M. Chow and O. Ng, "From technology adopters to creators: Leveraging AI-assisted vibe coding to transform clinical teaching and learning," *Med. Teacher*, vol. 47, no. 12, pp. 1927–1929, Apr. 2025, doi: 10.1080/0142159x.2025.2488353.

[14] E. Hutchins, *Cognition in the Wild*. Cambridge, MA, USA: MIT Press, 1996.

[15] D. Dellermann, P. Ebel, M. Söllner, and J. M. Leimeister, "Hybrid intelligence," *Bus. Inf. Syst. Eng.*, vol. 61, no. 5, pp. 637–643, Oct. 2019, doi: 10.1007/s12599-019-00595-2.

[16] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, "A survey on large language models for code generation," 2024, *arXiv:2406.00515*.

[17] T. Weber, M. Brandmaier, A. Schmidt, and S. Mayer, "Significant productivity gains through programming with large language models," *Proc. ACM Human-Comput. Interact.*, vol. 8, no. EICS, pp. 1–29, Jun. 2024, doi: 10.1145/3661145.

[18] L. Gao, S. Biderman, S. Black, L. Golding, T. Hoppe, C. Foster, J. Phang, H. He, A. Thite, N. Nabeshima, S. Presser, and C. Leahy, "The pile: An 800GB dataset of diverse text for language modeling," 2021, *arXiv:2101.00027*.

[19] T. Haigh and P. E. Ceruzzi, *A New History of Modern Computing*. Cambridge, MA, USA: MIT Press, 2021.

[20] T. Haigh, M. Priestley, and C. Rope, *ENIAC in Action: Making and Remaking the Modern Computer*. Cambridge, MA, USA: MIT Press, 2016.

[21] G. O'Regan, *A Brief History of Computing*. Cham, Switzerland: Springer, 2021.

[22] R. Rojas, *Konrad Zuse's Early Computers: The Quest for the Computer in Germany*. Cham, Switzerland: Springer, 2023.

[23] P. E. Ceruzzi, *A History of Modern Computing*. Cambridge, MA, USA: MIT Press, 2012.

[24] M. V. Wilkes, D. J. Wheeler, and S. Gill, *The Preparation of Programs for an Electronic Digital Computer*. Reading, MA, USA: Addison-Wesley, 1957.

[25] D. Camarmas-Alonso, F. Garcia-Carballeira, A. Calderon-Mateos, and E. del-Pozo-Puñal, "CREATOR: An educational integrated development environment for RISC-V programming," *IEEE Access*, vol. 12, pp. 127702–127717, 2024, doi: 10.1109/ACCESS.2024.3406935.

[26] T. Newhall, K. C. Webb, I. Romea, E. Stavis, and S. J. Matthews, "ASM visualizer: A learning tool for assembly programming," in *Proc. 56th ACM Tech. Symp. Comput. Sci. Educ.*, Pittsburgh, PA, USA, Feb. 2025, pp. 840–846, doi: 10.1145/3641554.3701793.

[27] M. Campbell-Kelly, W. Aspray, N. Ensmenger, and J. R. Yost, *Computer: A History of the Information Machine*. New York, NY, USA: Routledge, 2018.

[28] C. Böhm and G. Jacopini, "Flow diagrams, Turing machines and languages with only two formation rules," *Commun. ACM*, vol. 9, no. 5, pp. 366–371, May 1966, doi: 10.1145/355592.365646.

[29] J. E. Sammet, *Programming Languages: History and Fundamentals*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1969.

[30] D. Nofre, "A compelling image: The tower of Babel and the proliferation of programming languages during the 1960s," *IEEE Ann. Hist. Comput.*, vol. 47, no. 1, pp. 22–35, Jan. 2025, doi: 10.1109/MAHC.2024.3450530.

[31] E. W. Dijkstra, "Letters to the editor: Go to statement considered harmful," *Commun. ACM*, vol. 11, no. 3, pp. 147–148, Mar. 1968, doi: 10.1145/362929.362947.

[32] E. W. Dijkstra, "The humble programmer," *Commun. ACM*, vol. 15, no. 10, pp. 859–866, Oct. 1972, doi: 10.1145/355604.361591.

[33] A. Raheem and H. Azmat, "Declarative power: The enduring relevance of SQL in the age of big data," *EuroVantage J. Artif. Intell.*, vol. 2, no. 2, pp. 64–71, Apr. 2025.

[34] S. W. Loke, "Declarative programming of integrated peer-to-peer and Web based systems: The case of prolog," *J. Syst. Softw.*, vol. 79, no. 4, pp. 523–536, Apr. 2006, doi: 10.1016/j.jss.2005.04.005.

[35] K. Satoh, "PROLEG: Practical legal reasoning system," in *Prolog: The Next 50 Years*, D. S. Warren, V. Dahl, T. Eiter, M. V. Hermenegildo, R. Kowalski, F. Rossi, Eds., Cham, Switzerland: Springer, 2023, pp. 277–283.

[36] H. Princis, C. David, and A. Mycroft, "Enhancing SQL query generation with neurosymbolic reasoning," in *Proc. 39th Annu. AAAI Conf. Artif. Intell.*, Philadelphia, PA, USA, Feb. 2024, pp. 19959–19968, doi: 10.1609/aaai.v39i19.34198.

[37] J. W. Lloyd, "Practical advantages of declarative programming," in *Proc. Joint Conf. Declerative Program. GULP-PRODE*, Sep. 1994, pp. 3–17.

[38] A. Silberschatz, H. F. Korth, and S. Sudarshan, *Database System Concepts*. New York, NY, USA: McGraw-Hill Education, 2020.

[39] W. Drabent, "The prolog debugger and declarative programming," in *Proc. 29th Int. Symp. Log.-Based Program Synth. Transform.*, Porto, Portugal, Oct. 2020, pp. 193–208, doi: 10.1007/978-3-030-45260-5_12.

[40] J. Hughes, "Why functional programming matters," *Comput. J.*, vol. 32, no. 2, pp. 98–107, Feb. 1989, doi: 10.1093/comjnl/32.2.98.

[41] P. Wadler, "The essence of functional programming," in *Proc. 19th ACM SIGPLAN-SIGACT Symp. Princ. Program. Lang.-POPL*, Albuquerque, NM, USA, Jan. 1992, pp. 1–14, doi: 10.1145/143165.143169.

[42] B. Meyer, *Object-Oriented Software Construction*. Englewood Cliffs, NJ, USA: Prentice-Hall, 1997.

[43] A. C. Kay, "The early history of smalltalk," in *History of Programming Languages—II*. New York, NY, USA: Association for Computing Machinery, Jan. 1996, pp. 511–598, doi: 10.1145/234286.1057828.

[44] B. Stroustrup, *The Design and Evolution of C++*. Reading, MA, USA: Addison-Wesley, 1995.

[45] E. A. Nyamsi, *Programming4Modeling: Codes in Modellen Auf Basis Von Java Und UML*. Wiesbaden, Germany: Springer, 2025.

[46] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1995.

[47] E. Dickhaut, A. Janson, M. Söllner, and J. M. Leimeister, "Lawfulness by design–development and evaluation of lawful design patterns to consider legal requirements," *Eur. J. Inf. Syst.*, vol. 33, no. 4, pp. 441–468, Jul. 2024, doi: 10.1080/0960085x.2023.2174050.

[48] H. J. W. Percival and B. Gregory, *Architecture Pattern With Python*. Sebastopol, CA, USA: O'Reilly Media, 2020.

[49] R. C. Martin, *Clean Architecture: A Craftsman's Guide To Software Structure and Design*. Boston, MA, USA: Prentice-Hall, 2018.

[50] C. Elliott, V. Vijayakumar, W. Zink, and R. Hansen, "National instruments LabVIEW: A programming environment for laboratory automation and measurement," *J. Assoc. Lab. Autom.*, vol. 12, no. 1, pp. 17–24, Feb. 2007, doi: 10.1016/j.jala.2006.07.012.

[51] K. N. Whitley and A. F. Blackwell, "Visual programming in the wild: A survey of LabVIEW programmers," *J. Vis. Lang. Comput.*, vol. 12, no. 4, pp. 435–472, Aug. 2001, doi: 10.1006/jvlc.2000.0198.

[52] G. Fraser, U. Heuer, N. Korber, F. Obermuller, and E. Wasmeier, "Litter-Box: A linter for scratch programs," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Eng., Softw. Eng. Educ. Training (ICSE-SEET)*, Madrid, Spain, May 2021, pp. 183–188, doi: 10.1109/ICSE-SEET52601.2021.00028.

[53] J. Maloney, M. Resnick, N. Rusk, B. Silverman, and E. Eastmond, "The scratch programming language and environment," *ACM Trans. Comput. Educ.*, vol. 10, no. 4, pp. 1–15, Nov. 2010, doi: 10.1145/1868358.1868363.

[54] A. Bucaioni, A. Cicchetti, and F. Ciccozzi, "Modelling in low-code development: A multi-vocal systematic review," *Softw. Syst. Model.*, vol. 21, no. 5, pp. 1959–1981, Oct. 2022, doi: 10.1007/s10270-021-00964-0.

[55] F. Mohammed Ozman, "A systematic literature review on current developments of low code-no code solutions in the IT sector," *World J. Adv. Eng. Technol. Sci.*, vol. 14, no. 3, pp. 162–169, Mar. 2025, doi: 10.30574/wjaets.2025.14.3.0072.

[56] A. J. Ko, R. Abraham, L. Beckwith, A. Blackwell, M. Burnett, M. Erwig, C. Scaffidi, J. Lawrance, H. Lieberman, B. Myers, M. B. Rosson, G. Rothermel, M. Shaw, and S. Wiedenbeck, "The state of the art in end-user software engineering," *ACM Comput. Surv.*, vol. 43, no. 3, pp. 1–44, Apr. 2011, doi: 10.1145/1922649.1922658.

[57] M. Allamanis, E. T. Barr, P. Devanbu, and C. Sutton, "A survey of machine learning for big code and naturalness," *ACM Comput. Surv.*, vol. 51, no. 4, pp. 1–37, Jul. 2018, doi: 10.1145/3212695.

[58] A. Hindle, E. T. Barr, M. Gabel, Z. Su, and P. Devanbu, "On the naturalness of software," *Commun. ACM*, vol. 59, no. 5, pp. 122–131, Apr. 2016, doi: 10.1145/2902362.

[59] M. R. Lyu, "AI techniques in software engineering paradigm," in *Proc. ACM/SPEC Int. Conf. Perform. Eng.*, Berlin, Germany, Mar. 2018, p. 2, doi: 10.1145/3184407.3184440.

[60] M. Bruch, M. Monperrus, and M. Mezini, "Learning from examples to improve code completion systems," in *Proc. 7th Joint Meeting Eur. Softw. Eng. Conf. ACM SIGSOFT Symp. Found. Softw. Eng.*, Aug. 2009, pp. 213–222, doi: 10.1145/1595696.1595728.

[61] S. Proksch, J. Lerch, and M. Mezini, "Intelligent code completion with Bayesian networks," *ACM Trans. Softw. Eng. Methodol.*, vol. 25, no. 1, pp. 1–31, Dec. 2015, doi: 10.1145/2744200.

[62] A. Svyatkovskiy, Y. Zhao, S. Fu, and N. Sundaresan, "Pythia: AI-assisted code completion system," in *Proc. 25th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, Anchorage, AK, USA, Jul. 2019, pp. 2727–2735, doi: 10.1145/3292500.3330699.

[63] S. Wang, J. Liu, Y. Qiu, Z. Ma, J. Liu, and Z. Wu, "Deep learning based code completion models for programming codes," in *Proc. 3rd Int. Symp. Comput. Sci. Intell. Control*, Sep. 2019, pp. 1–9, doi: 10.1145/3386164.3389083.

[64] A. Sergeyuk, Y. Golubev, T. Bryksin, and I. Ahmed, "Using AI-based coding assistants in practice: State of affairs, perceptions, and ways forward," *Inf. Softw. Technol.*, vol. 178, Feb. 2025, Art. no. 107610, doi: 10.1016/j.infsof.2024.107610.

[65] R. Ulfsnes, N. B. Moe, V. Stray, and M. Skarpen, "Transforming software development with generative AI: Empirical insights on collaboration and workflow," in *Generative AI for Effective Software Development*. Cham, Switzerland: Springer, Feb. 2024, pp. 219–234, doi: 10.1007/978-3-031-55642-5_10.

[66] A. Moradi Dakhel, V. Majdinasab, A. Nikanjam, F. Khomh, M. C. Desmarais, and Z. M. Jiang, "GitHub copilot AI pair programmer: Asset or liability?" *J. Syst. Softw.*, vol. 203, Sep. 2023, Art. no. 111734, doi: 10.1016/j.jss.2023.111734.

[67] A. Ziegler, E. Kalliamvakou, X. A. Li, A. Rice, D. Rifkin, S. Simister, G. Sittampalam, and E. Aftandilian, "Measuring GitHub Copilot's impact on productivity," *Commun. ACM*, vol. 67, no. 3, pp. 54–63, Feb. 2024, doi: 10.1145/3633453.

[68] J. T. Liang, M. Lin, N. Rao, and B. A. Myers, "Prompts are programs too! Understanding how developers build software containing prompts," *Proc. ACM Softw. Eng.*, vol. 2, no. FSE, pp. 1591–1614, Jun. 2025, doi: 10.1145/3729342.

[69] S. Willison. (Mar. 19, 2025). *Not All AI-Assisted Programming is Vibe Coding (But Vibe Coding Rocks)*. [Online]. Available: https://simonwillison.net/2025/Mar/19/vibe-coding/

[70] R. Sapkota, K. I. Roumeliotis, and M. Karkee, "Vibe coding vs. Agentic coding: Fundamentals and practical implications of agentic AI," 2025, *arXiv:2505.19443*.

[71] P. P. Ray, "A review on vibe coding: Fundamentals, state-of-the-art, challenges and future directions," *TechRxiv*, May 2025, doi: 10.36227/techrxiv.174681482.27435614/v1.

[72] *VIBE Definition & Meaning-Merriam-Webster*. Accessed: Oct. 22, 2025. [Online]. Available: https://www.merriam-webster.com/dictionary/vibe

[73] A. Gaggioli, E. Mazzoni, M. Benvenuti, C. Galimberti, A. Bova, E. Brivio, P. Cipresso, G. Riva, and A. Chirico, "Networked flow in creative collaboration: A mixed method study," *Creativity Res. J.*, vol. 32, no. 1, pp. 41–54, Jan. 2020, doi: 10.1080/10400419.2020.1712160.

[74] J.-F. Harvey, J. R. Cromwell, K. J. Johnson, and A. C. Edmondson, "The dynamics of team learning: Harmony and rhythm in teamwork arrangements for innovation," *Administ. Sci. Quart.*, vol. 68, no. 3, pp. 601–647, Sep. 2023, doi: 10.1177/00018392231166635.

[75] J. J. Cañas, M. T. Bajo, and P. Gonzalvo, "Mental models and computer programming," *Int. J. Hum.-Comput. Stud.*, vol. 40, no. 5, pp. 795–811, May 1994, doi: 10.1006/ijhc.1994.1038.

[76] L. Adie, F. van der Kleij, and J. Cumming, "The development and application of coding frameworks to explore dialogic feedback interactions and self-regulated learning," *Brit. Educ. Res. J.*, vol. 44, no. 4, pp. 704–723, Aug. 2018, doi: 10.1002/berj.3463.

[77] J. Sweller, "Cognitive load during problem solving: Effects on learning," *Cognit. Sci.*, vol. 12, no. 2, pp. 257–285, Jun. 1988, doi: 10.1016/0364-0213(88)90023-7.

[78] D. N. Rapp, "Mental models: Theoretical issues for visualizations in science education," in *Visualization in Science Education*. Dordrecht, The Netherlands: Springer, Mar. 2005, pp. 43–60, doi: 10.1007/1-4020-3613-2_4.

[79] D. A. Schön, *The Reflective Practitioner: How Professionals Think in Action*. London, U.K.: Routledge, 1992.

[80] J. Hollan, E. Hutchins, and D. Kirsh, "Distributed cognition: Toward a new foundation for human–computer interaction research," *ACM Trans. Comput.-Human Interact.*, vol. 7, no. 2, pp. 174–196, Jun. 2000, doi: 10.1145/353485.353487.

[81] J. S. Brown, A. Collins, and P. Duguid, "Situated cognition and the culture of learning," *Educ. Res.*, vol. 18, no. 1, pp. 32–42, Jan. 1989, doi: 10.3102/0013189X01800103.

[82] C. Meske, B. Abedin, M. Klier, and F. Rabhi, "Explainable and responsible artificial intelligence," *Electron. Markets*, vol. 32, no. 4, pp. 2103–2106, Dec. 2022, doi: 10.1007/s12525-022-00607-2.

[83] M. Csikszentmihalyi, *Flow: The Psychology of Optimal Experience*. New York, NY, USA: Harper & Row, 1990.

[84] P. H. D. S. Bermejo, A. L. Zambalde, A. O. Tonelli, S. A. Souza, L. A. Zuppo, and P. L. Rosa, "Agile principles and achievement of success in software development: A quantitative study in Brazilian organizations," *Proc. Technol.*, vol. 16, pp. 718–727, Oct. 2014, doi: 10.1016/j.protcy.2014.10.021.

[85] K. A. Ericsson, R. T. Krampe, and C. Tesch-Römer, "The role of deliberate practice in the acquisition of expert performance," *Psychol. Rev.*, vol. 100, no. 3, pp. 363–406, Jul. 1993, doi: 10.1037/0033-295x.100.3.363.

[86] M. Polanyi, *The Tacit Dimension*. Chicago, IL, USA: Univ. Chicago Press, 2009.

[87] Y. Ikutani, T. Kubo, S. Nishida, H. Hata, K. Matsumoto, K. Ikeda, and S. Nishimoto, "Expert programmers have fine-tuned cortical representations of source code," *Eneuro*, vol. 8, no. 1, pp. ENEURO.0405–20.2020, Dec. 2020, doi: 10.1523/eneuro.0405-20.2020.

[88] N. Assyne, H. Ghanbari, and M. Pulkkinen, "The state of research on software engineering competencies: A systematic mapping study," *J. Syst. Softw.*, vol. 185, Mar. 2022, Art. no. 111183, doi: 10.1016/j.jss.2021.111183.

[89] A. Matthias, "The responsibility gap: Ascribing responsibility for the actions of learning automata," *Ethics Inf. Technol.*, vol. 6, no. 3, pp. 175–183, Sep. 2004, doi: 10.1007/s10676-004-3422-1.

[90] P. M. Kuss and C. Meske, "From entity to relation? Agency in the era of artificial intelligence," *Commun. Assoc. Inf. Syst.*, vol. 56, pp. 633–674, Jun. 2025, doi: 10.17705/1cais.05626.

[91] R. Parasuraman and V. Riley, "Humans and automation: Use, misuse, disuse, abuse," *Human Factors*, vol. 39, no. 2, pp. 230–253, Jun. 1997, doi: 10.1518/001872097778543886.

[92] H. Naeem and J. Hauser, "Should we discourage AI extension? Epistemic responsibility and AI," *Philosophy Technol.*, vol. 37, no. 3, Jul. 2024, Art. no. 91, doi: 10.1007/s13347-024-00774-4.

[93] A. Gadde, "Democratizing software engineering through generative AI and vibe coding: The evolution of no-code development," *J. Comput. Sci. Technol. Stud.*, vol. 7, no. 4, pp. 556–572, May 2025, doi: 10.32996/jcsts.2025.7.4.66.

[94] A. Beheshti, "Natural language-oriented programming (NLOP): Towards democratizing software creation," in *Proc. IEEE Int. Conf. Softw. Services Eng. (SSE)*, Shenzhen, China, Jul. 2024, pp. 258–267, doi: 10.1109/SSE62657.2024.00047.

[95] S. Ma, J. Wang, Y. Zhang, X. Ma, and A. Y. Wang, "DBox: Scaffolding algorithmic programming learning through learner-LLM co-decomposition," in *Proc. CHI Conf. Human Factors Comput. Syst.*, Yokohama, Japan, Apr. 2025, pp. 1–20, doi: 10.1145/3706598.3713748.

[96] T. Li, T. Maheshwari, and A. Voelker, "User-centered design with AI in the loop: A case study of rapid user interface prototyping with 'vibe coding,'" 2025, *arXiv:2507.21012*.

[97] J. Ma, K. Sreedhar, V. Liu, S. Wang, P. A. Perez, and L. B. Chilton, "DIDUP: Dynamic iterative development for UI prototyping," 2024, *arXiv:2407.08474*.

[98] Z. Wu, D. Ji, X. Zeng, D. Wu, and M. Shidujaman, "AI creativity and the human-AI co-creation model," in *Human–Computer Interaction. Theory, Methods and Tools (HCII 2021)*. Cham, Switzerland: Springer, Jul. 2021, doi: 10.1007/978-3-030-78462-1_13.

[99] H. Subramonyam, D. Thakkar, A. Ku, J. Dieber, and A. K. Sinha, "Prototyping with prompts: Emerging approaches and challenges in generative AI design for collaborative software teams," in *Proc. CHI Conf. Human Factors Comput. Syst.*, Yokohama, Japan, Apr. 2025, pp. 1–22, doi: 10.1145/3706598.3713166.

[100] L. Banh, F. Holldack, and G. Strobel, "Copiloting the future: How generative AI transforms software engineering," *Inf. Softw. Technol.*, vol. 183, Jul. 2025, Art. no. 107751, doi: 10.1016/j.infsof.2025.107751.

[101] P. Pajo, "Vibe coding: Revolutionizing software development with AI-generated code," De La Salle-College of Saint Benilde, Manila, Philippines, Tech. Rep., 2025, doi: 10.13140/RG.2.2.36458.22727.

[102] G. D. De Siano, A. R. Fasolino, G. Sperlí, and A. Vignali, "Translating code with large language models and human-in-the-loop feedback," *Inf. Softw. Technol.*, vol. 186, Oct. 2025, Art. no. 107785, doi: 10.1016/j.infsof.2025.107785.

[103] M. Akhoroz and C. Yildirim, "Conversational AI as a coding assistant: Understanding programmers' interactions with and expectations from large language models for coding," 2025, *arXiv:2503.16508*.

[104] S. Suh, M. Lai, K. Pu, S. P. Dow, and T. Grossman, "StoryEnsemble: Enabling dynamic exploration & iteration in the design process with AI and forward-backward propagation," 2025, *arXiv:2508.03182*.

[105] N. Li, H. Zhou, and K. Mikel-Hong, "Generative AI enhances team performance and reduces need for traditional teams," 2024, *arXiv:2405.17924*.

[106] D. Mullens and S. Shen, "2ACT: AI-accentuated career transitions via skill bridges," 2025, *arXiv:2505.07914*.

[107] C. Mackinnon and M. E. Foster, "Lowering the barrier: Conversational interfaces as a novice-friendly path to data visualisation," in *Proc. 7th ACM Conf. Conversational User Interface*, Waterloo, ON, Canada, Jul. 2025, pp. 1–7, doi: 10.1145/3719160.3737623.

[108] M. Gindert and M. Lutz Müller, "The impact of generative artificial intelligence on ideation and the performance of innovation teams (preprint)," 2024, *arXiv:2410.18357*.

[109] K. Crowston and F. Bolici, "Deskilling and upskilling with AI system," *Inf. Res. Int. Electron. J.*, vol. 30, no. 1, pp. 1009–1023, Mar. 2025, doi: 10.47989/ir30iConf47143.

[110] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do users write more insecure code with AI assistants?" 2022, *arXiv:2211.03622*.

[111] S. Kumar Navneet and J. Chandra, "Rethinking autonomy: Preventing failures in AI-driven software engineering," 2025, *arXiv:2508.11824*.

[112] H. Le, "The deskilling of software development and the impact of AI chatbots on programmers' skills and roles," in *Generative AI in Software Engineering*. Hershey, PA, USA: IGI Global Scientific Publishing, 2025, pp. 397–426.

[113] C. Knobel and N. Radziwill, "Vibe coding: Is human nature the ghost in the machine?" 2025, *arXiv:2508.20918*.

[114] J. Bogner, R. Verdecchia, and I. Gerostathopoulos, "Characterizing technical debt and antipatterns in AI-based systems: A systematic mapping study," in *Proc. IEEE/ACM Int. Conf. Tech. Debt (TechDebt)*, Madrid, Spain, May 2021, pp. 64–73, doi: 10.1109/TECHDEBT52882.2021.00016.

[115] R. Moretti. *Vulnerability Uncovered In Wix Vibe Coding Platform*. Accessed: Oct. 22, 2025. [Online]. Available: https://www.searchenginejournal.com/vulnerability-uncovered-in-wix-vibe-coding-platform/552554/

[116] (2025). *GenAI Code Security Report*. [Online]. Available: https://www.veracode.com/resources/analyst-reports/2025-genai-code-security-report/

[117] D. Cotroneo, C. Improta, and P. Liguori, "Human-written vs. AI-generated code: A large-scale study of defects, vulnerabilities, and complexity," 2025, *arXiv:2508.21634*.

[118] Z. Porter, P. Ryan, P. Morgan, J. Al-Qaddoumi, B. Twomey, P. Noordhof, J. McDermid, and I. Habli, "Unravelling responsibility for AI," 2023, *arXiv:2308.02608*.

[119] F. Geng, A. Shah, H. Li, N. Mulla, S. Swanson, G. S. Raj, D. Zingaro, and L. Porter, "Exploring student-AI interactions in vibe coding," 2025, *arXiv:2507.22614*.

[120] Aarti, "Generative AI in software development : An overview and evaluation of modern coding tools," *Int. J. Multidisciplinary Res.*, vol. 6, no. 3, Jun. 2024, Art. no. IJFMR240323271, doi: 10.36948/ijfmr.2024.v06i03.23271.

[121] H. S. Alwageed and R. Ahmad Khan, "The role of generative AI in strengthening secure software coding practices: A systematic perspective," 2025, *arXiv:2504.19461*.

[122] P. Mäder and A. Egyed, "Assessing the effect of requirements traceability for software maintenance," in *Proc. 28th IEEE Int. Conf. Softw. Maintenance (ICSM)*, Trento, Italy, Sep. 2012, pp. 171–180, doi: 10.1109/ICSM.2012.6405269.

[123] S. Casper et al., "Black-box access is insufficient for rigorous AI audits," in *Proc. ACM Conf. Fairness, Accountability, Transparency*, Jun. 2024, pp. 2254–2272, doi: 10.1145/3630106.3659037.

[124] F. Gu, Z. Liang, H. Li, and J. Ma, "The Matthew effect of AI programming assistants: A hidden bias in software evolution," 2025, *arXiv:2509.23261*.

[125] S.-C. Necula, "Artificial intelligence impact on the labour force - searching for the analytical skills of the future software engineers," 2023, *arXiv:2302.13229*.

[126] F. Trotta. (Apr. 2, 2025). *5 Vibe Coding Risks and Ways To Avoid Them in 2025*. [Online]. Available: https://zencoder.ai/blog/vibe-coding-risks

[127] K. Ferdowsi, R. Huang, M. B. James, N. Polikarpova, and S. Lerner, "Validating AI-generated code with live programming," in *Proc. CHI Conf. Human Factors Comput. Syst.*, Honolulu, HI, USA, May 2024, pp. 1–8, doi: 10.1145/3613904.3642495.

[128] C. Gao, X. Hu, S. Gao, X. Xia, and Z. Jin, "The current challenges of software engineering in the era of large language models," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 5, pp. 1–30, May 2025, doi: 10.1145/3712005.

[129] D. Hu, P. Santiesteban, M. Endres, and W. Weimer, "Towards a cognitive model of dynamic debugging: Does identifier construction matter?" *IEEE Trans. Softw. Eng.*, vol. 50, no. 11, pp. 3007–3021, Nov. 2024, doi: 10.1109/TSE.2024.3465222.

CHRISTIAN MESKE is a Full Professor of socio-technical system design and artificial intelligence with the Faculty of Mechanical Engineering and the Institute of Work Science, Ruhr University Bochum, Germany. He has contributed to numerous interdisciplinary research projects and published more than 80 articles in leading conferences and journals, including *Applied Energy*, *Business and Information Systems Engineering*, *Communications of the Association for Information Systems*, *Information Systems Journal*, *Information Systems Management*, *Journal of the Association for Information Systems*, *MIS Quarterly Executive*, and *Solar Energy*. He also serves as a Senior Editor for *Information Systems Management*, an Associate Editor for *Behaviour and Information Technology*, and holds various other roles.

TOBIAS HERMANNS received the M.Sc. degree in applied computer science from Ruhr University Bochum, Germany. He is currently pursuing the Ph.D. degree with the Chair of Socio-technical System Design and Artificial Intelligence, Faculty of Mechanical Engineering and the Institute of Work Science, and affiliated with the Faculty of Computer Science with Ruhr University Bochum. In parallel to his studies, he worked at the Chair of Embedded Security in collaboration with the Max Planck Institute for Security and Privacy, contributing to the development and maintenance of "HAL–The Hardware Analyzer", an open-source tool for reverse engineering hardware, used in both research and teaching. His research focuses on human–computer interaction and user experience in the context of generative artificial intelligence, especially in relation to interaction design with large language models and the dynamics of human–AI collaboration. His interests include human–computer interaction, human–AI interaction, Generative AI, and socio-technical system design.

ESTHER VON DER WEIDEN received the M.Sc. degree in applied computer science from the University of Duisburg–Essen, Duisburg, Germany. She is currently pursuing the Ph.D. degree with the Chair of Socio-Technical System Design and Artificial Intelligence, Data Protection Office, Ruhr University Bochum, Bochum, Germany. She is currently a Research Associate with the Chair of Socio-Technical System Design and Artificial Intelligence, Data Protection Office, Ruhr University Bochum. Her research focuses on the intersection of data protection and artificial intelligence. Her research interests include data protection, generative AI, information engineering, AI, and information literacy.

KAI-UWE LOSER received the Ph.D. degree in computer science from Technical University Dortmund, Dortmund, Germany. He is also a Certified Data Protection Auditor. He serves as the Official Data Protection Officer with Ruhr University Bochum and the University Duisburg–Essen, and is Vice President of the Professional Association of Data Protection Officers (BvD). He has several publications in peer-reviewed journals and international conference proceedings. His research interests include data protection, modeling of socio-technical systems, knowledge management and learning organizations, and the application of groupware to support knowledge management and learning organizations.

THORSTEN BERGER is currently a Professor in computer science with Ruhr University Bochum, Germany. He published more than 180 papers, many in A* venues, such as ICSE; FSE, and ASE, and Q1 journals such as IEEE TSE. He is the co-author of the textbook *Domain-Specific Languages: Effective Modeling, Automation, and Reuse*. His research focuses on AI engineering, software variability, model-driven engineering, and software security. He received competitive grants from Swedish Research Council, Wallenberg Autonomous Systems Program, Vinnova Sweden (EU ITEA), and European Union, as well as he was awarded the Fellowship of the Royal Swedish Academy of Sciences and of the Wallenberg Foundation—one of the highest recognitions for researchers in Sweden. He received three most-influential-paper, two best-paper, and three distinguished-reviewer awards.